

# Audit Intake & Planning MVP — Product Specification

*Prepared for: a CPA-led MVP, positioned for small/mid-size firms doing not-for-profit audits (churches, schools, nonprofits with program income). Grounded in real KBA/AICPA planning practice (client acceptance factors, initial-engagement opening-balance procedures, entity/environment understanding, revenue-cycle understanding) so the request-list logic reflects how audits actually get planned, not generic onboarding.*

---

## 1. Clarifying Questions — Resolved

Your answers below replace my earlier placeholder assumptions. This changes the scope in one important way (document exchange is now in v1, not deferred) — flagged clearly where it affects other sections.

1. **Who fills out the intake?** The finance department (if the entity has one separate from operations) or an NFP administrator/director who understands NFP accounting and financial statements. **Implication:** the respondent is more financially literate than a generic "any admin" persona — question wording can use standard NFP terminology (net assets, restricted contributions, program service revenue) rather than needing to over-simplify everything, though branching/plain-language structure still matters for less-technical respondents at smaller entities.
2. **Single-firm or multi-tenant?** Single firm, to build and prove the prototype first. The data model still carries `firm_id` on the relevant tables (cheap to include now, expensive to retrofit later) but there's no need to build firm signup/onboarding, billing, or a firm-admin screen for v1 — one firm is seeded/hardcoded.
3. **In-app list only, or with document exchange? Both — this is a portal.** In-app request list *and* a document exchange feature where the client uploads files against specific request-list items and the auditor tracks receipt status. This is a meaningful scope increase from my original deferral — see the revised Section 3 and Section 7 below.
4. **Role separation on day one?** Confirmed single "firm user" role for MVP; partner/senior sign-off stays deferred to v2, field reserved in the data model as originally planned.

Everything below reflects these answers.

---

## 1.5 Audit Methodology Primer — read this before Section 7 (for the developer, who isn't an auditor)

This section didn't exist in earlier drafts of this spec. It needs to, because the data model and logic engine below only make sense with this context — without it, the schema looks like arbitrary tables instead of a direct translation of how audits are actually planned.

**The methodology this product automates is called KBA (Knowledge-Based Audit).** It's a real, published methodology (not something invented for this product) made up of hundreds of numbered reference documents — the CPA firm licenses access to the full set and uses them as the standard playbook for every audit. Every document has a prefix and a number, and the number tells you which phase of the audit it belongs to:

| Number range | Phase                  | What it covers                                                 |
|--------------|------------------------|----------------------------------------------------------------|
| 000s         | Resources              | Reference material, not filled out per-client                  |
| 100s         | Communication Hub      | Cross-team notes during the audit                              |
| 200s         | Preliminary Engagement | Client acceptance, engagement letters, opening balances        |
| 300s         | Entity Understanding   | Who the client is, how they operate, fraud risk, estimates     |
| 400s         | Design of Controls     | How the client's own processes work, cycle by cycle            |
| 500s         | Assessing Risk         | Formal risk documentation (auditor judgment, not client input) |
| 600s         | Designing Procedures   | Which audit programs to run (auditor judgment)                 |
| 700s         | Tests of Controls      | Fieldwork execution                                            |
| 800s         | Substantive Procedures | Fieldwork execution — this is what the request list feeds      |

| Number range | Phase                    | What it covers                          |
|--------------|--------------------------|-----------------------------------------|
| 900s         | Evaluate/Conclude/Report | Forming the opinion, issuing the report |

**The product only touches the 200s, 300s, and 400s.** Phases 500–900 are the auditor's actual judgment and fieldwork — no questionnaire replaces those, and the product shouldn't try to. What the product *does* do is replace the manual, error-prone version of gathering facts for the 200s/300s/400s forms and deciding which of the 400s "transaction control" forms even apply to this specific client.

**The one concept that matters most: scoping.** One specific form, **KBA-400**, is a checklist of every possible audit area a nonprofit might have (Cash, Investments, Revenue, Payroll, PP&E, Debt, and about 15 others). For each area, the auditor decides: *is this significant for this client?* If yes, a specific 400-series "transaction controls" form gets filled out for it (e.g., KBA-403 for Revenue, KBA-409 for Treasury/Investments), which in turn drives which documents get requested from the client. **If no, that entire form — and every request-list item tied to it — is skipped. Not deprioritized. Skipped.** A school with no investments never gets asked about investment controls, never gets a Treasury workpaper prepared, never gets a request-list item for a brokerage statement. That's not a shortcut the software takes — it's exactly how a human auditor is supposed to work, and it's the reason the product has real value: doing this scoping decision by hand, correctly, for every client, every year, is exactly the tedious, error-prone step this tool exists to standardize.

**Worked example (the one used throughout this spec and the prototype):** Whitfield Preparatory Academy, a school, re-audited after a 2-year gap (last audited FY2023, current audit FY2026). See Section 8 for the full scoping table and Section 13 for how it plays out end-to-end. The prototype file ([audit-portal-prototype.html](#)) shows this same example as a clickable mockup — the Scoping tab there lets you toggle "Investments: Yes/No" and watch the Transaction Controls list below it switch KBA-409 between Included and Skipped live. That interaction *is* the scoping mechanic described above, made visible.

---

## 2. Core Product Definition

**What it is:** A web app where a CPA firm sends a branching, plain-language planning questionnaire to a nonprofit audit client (church, school, or program-income nonprofit), and the client's answers are automatically converted into a tailored initial document request list plus a set of "auditor follow-up" flags — replacing the manual, inconsistent process of a senior emailing a generic checklist and then chasing clarifications.

**Who it's for:** Audit seniors/managers at small-to-mid CPA firms (and eventually internal audit teams) who run repeatable planning intake across many similar engagements per year.

**Painful workflow it solves:** Today, tailoring the request list and figuring out what to ask a client requires a human to remember firm knowledge (what's different about a first-year audit vs. a recurring one, what a church needs vs. a school) and manually build the list each time. This is slow, inconsistent between staff, and gives the client a worse first impression.

**Main outcome for the user:** In minutes instead of an intake meeting + follow-up emails, the auditor gets a structured, entity-type-aware, engagement-type-aware request list and a short "things to clarify before fieldwork" flag list — ready to send or use in planning.

---

### 3. MVP Scope

#### Must-have (v1 — the thing you ship)

- Firm user login (single role, single firm)
- Create an engagement (client name, entity type, client type/history, fiscal year end)
- Choose/apply a question template (seeded: NFP-general, church, school, NFP-with-program-income)
- Branching questionnaire, respondent-facing, mobile-friendly, using standard NFP terminology (your respondents are finance-literate — see Section 1)
- Answer types: yes/no, single-select, multi-select, short text, number, date
- Rules engine: answers → request-list items + follow-up flags, **each item carrying an embedded guidance note visible only to the firm user** (why it's requested / what it should include — internal rationale, not shown to the client) — deterministic, not AI
- Auditor summary/review screen: request list grouped by planning area, flags highlighted
- **Document exchange (portal, simplified for v1):** respondent can upload one or more files against each request-list item; each item has a status (Requested / Uploaded / Received / Needs Follow-up); auditor sees upload status per item in the review screen
- Export request list (PDF and CSV)
- Reusable templates (clone/edit an existing template into a new one)

#### Should-have (fast follow, weeks 3–6, not blocking v1 launch)

- Save-and-resume for the respondent (long questionnaires abandoned otherwise)
- Auditor can override/add/remove request-list items before finalizing (auditors will never trust a fully automated list — let them edit it)
- Simple engagement dashboard listing all active intakes and their status (not started / in progress / complete)

- Basic branching preview ("test as respondent") for the firm user building/editing a template
- Comment/note thread per request-list item (respondent and auditor can leave a short note, e.g., "this doesn't apply to us")

### Later-version (v2+, deliberately deferred)

- Partner/senior role separation and sign-off workflow
- Document versioning (re-uploads replacing prior files with history) and e-signature
- AI-generated planning memo draft from answers
- Multi-firm onboarding/admin (billing, firm signup) — not needed while proving with one firm
- Multi-firm marketplace of templates
- Integrations (QuickBooks, engagement software, e-signature)
- Analytics across engagements (e.g., "average time to complete intake")

### Explicitly excluded (not on any near-term roadmap — say so if asked)

- Full working paper / trial balance / sampling functionality (this is not Caseware/AuditBoard)
- Billing/time tracking
- Risk assessment memo automation beyond the request list + flags
- Multi-industry expansion (stay in NFP until the workflow is proven)

**Note on document exchange scope:** you've confirmed this needs to be a real portal, not just an export — so file upload/storage moves from "deferred" to "must-have," but kept deliberately simple: one or a few files per request item, a status field, no versioning, no e-signature, no in-app collaborative editing. That's enough to prove the portal loop (request → upload → auditor sees it) without building the heavier document-management features a full portal eventually needs.

## 4. User Types and Jobs To Be Done

| User                       | Who they are                           | What they're trying to accomplish                                                           | Pain point the MVP reduces                                                          |
|----------------------------|----------------------------------------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Firm user (auditor)</b> | Senior/manager/partner at the CPA firm | Get a tailored request list out the door fast, without reinventing it per client; make sure | Manual list-building, inconsistency between staff, forgetting entity-specific items |

| User                               | Who they are                                                                                                                                                    | What they're trying to accomplish                                                                                                                                                          | Pain point the MVP reduces                                                                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|                                    |                                                                                                                                                                 | nothing entity-specific is missed                                                                                                                                                          | (e.g., in-kind contributions for a church, tuition/Title funds for a school)                                                                       |
| <b>Respondent (client contact)</b> | The entity's finance department contact (where one exists separately), or an NFP administrator/director who understands NFP accounting and financial statements | Answer questions using terminology they're already familiar with, understand <i>why</i> each item is being requested (via embedded guidance), and upload supporting documents in one place | One-size-fits-all checklists with irrelevant items; not knowing why something is being asked; no single place to submit documents against the list |
| <b>(v2) Partner</b>                | Reviews and signs off                                                                                                                                           | Confirm the request list and flags reflect proper risk judgment before it goes out                                                                                                         | Not in v1 — noted so the data model doesn't block it later                                                                                         |

## 5. User Flow (end-to-end)

1. **Firm user logs in** → sees engagement dashboard.
2. **Firm user creates an engagement:** client name, fiscal year end, **entity type** (church / school / NFP with program income / general NFP), **client type/history** (first-year never audited / year-over-year continuing / re-audit after a gap / first-year with predecessor auditor).
3. **System auto-selects the matching template** (or firm user picks manually) based on entity type + client type combination — e.g., "NFP-general + first-year-with-predecessor" pulls in the base question set *plus* the predecessor-auditor communication block *plus* opening-balance questions.
4. **Firm user sends the intake link** (or opens it themselves to fill in during a call).
5. **Respondent opens the link**, answers branching questions in plain language, can save and resume (should-have).
6. **Respondent submits.**

7. **System evaluates answers against the rules engine:** generates request-list items, tags each with a planning area (e.g., Cash, Contributions/Revenue, Payroll, PP&E, IT/General Controls, Governance), and raises follow-up flags where answers indicate risk, missing info, or "client doesn't know."
  8. **Firm user opens the auditor review screen:** sees the full request list grouped by area, sees flags separately with the reason each was raised, can edit/add/remove items (should-have), and finalizes the list.
  9. **The finalized request list becomes visible to the respondent in the portal** — each item shows its guidance note (why it's requested) and an upload control.
  10. **Respondent returns to the portal over time and uploads documents** against individual items; each item's status updates (Requested → Uploaded → Received once the auditor confirms it, or Needs Follow-up if the auditor flags it back).
  11. **Firm user monitors document status** from the auditor review screen alongside the original request list — no separate tool needed to track what's come in.
  12. **Firm user can still export** the request list (PDF/CSV) for use outside the tool (e.g., pasting into planning memo software) at any point.
  13. **Next action:** firm user proceeds to planning/fieldwork once enough documents are received; engagement status updates to "intake complete" on the dashboard.
- 

## 6. Screens / Pages

### 6.1 Login / Auth

- **Purpose:** firm user access
- **Components:** email/password (or magic link) form
- **Actions:** log in, reset password
- **Data shown:** none

### 6.2 Engagement Dashboard

- **Purpose:** firm-wide list of engagements and their intake status
- **Components:** table (client name, entity type, status, last updated), "New Engagement" button, search/filter by status
- **Actions:** create engagement, open engagement, resend intake link
- **Data shown:** engagement name, status (Draft / Sent / In Progress / Submitted / Reviewed), fiscal year end

### 6.3 New Engagement Setup

- **Purpose:** capture engagement metadata and select template

- **Components:** client name field, fiscal year end date picker, entity type selector, client type/history selector, template auto-suggested (editable dropdown)
- **Actions:** save draft, generate intake link
- **Data shown:** the auto-suggested template name and a short description of what it includes

## 6.4 Template Library (firm-side)

- **Purpose:** manage reusable question sets
- **Components:** list of templates (seeded + custom), "Clone template" and "New template" buttons
- **Actions:** view, clone, edit, archive a template
- **Data shown:** template name, entity type(s)/client type(s) it applies to, number of questions, last edited

## 6.5 Template Builder / Editor

- **Purpose:** build or edit a question set and its branching + rule mappings
- **Components:** question list (drag to reorder), add-question form (text, type, options, branch conditions), rule mapping panel (which answer → which request-list item / flag), "Preview as respondent" toggle
- **Actions:** add/edit/delete question, set branch logic, map answer to rule, preview, publish
- **Data shown:** live preview of the respondent view

## 6.6 Respondent Intake Form (portal login required — this is a portal, not an anonymous public link)

- **Purpose:** the client-facing questionnaire
- **Components:** progress bar, one section/planning-area at a time, NFP-terminology question text, appropriate input control per answer type, "save and continue later" link (should-have)
- **Actions:** answer, go back, submit
- **Data shown:** only the questions relevant given branching so far — never a static full list

## 6.7 Auditor Review / Request List Screen

- **Purpose:** the core output screen — what the auditor actually works from
- **Components:** request list grouped by planning area (collapsible sections), each item shows the source answer that triggered it, its internal guidance note (auditor-only rationale), plus its **document status** (Requested/Uploaded/Received/Needs Follow-up); separate "Flags" panel showing follow-up items with reason and severity (e.g., "Missing information," "Elevated risk," "Client unsure"); edit controls per item (should-have); export buttons; per-item link to view/download uploaded files



- **Actions:** review, edit list (should-have), mark an uploaded item "Received" or "Needs Follow-up", export PDF/CSV, mark engagement "reviewed"
- **Data shown:** full generated list + flags + raw respondent answers (expandable, for traceability) + document upload status

## 6.9 Client Document Portal (respondent-facing, post-finalization)

- **Purpose:** where the respondent comes back to see the finalized request list and upload documents against it
- **Components:** request list grouped by planning area, each item showing its plain description (what to upload — *no internal guidance note, that stays auditor-only*), an upload control, and its current status
- **Actions:** upload file(s) per item, leave a note per item (should-have), view status
- **Data shown:** finalized request list with per-item upload status — this screen only appears once the auditor has finalized the list (step 8–9 in the flow)

## 6.8 Engagement Detail (overview)

- **Purpose:** single source of truth for one engagement
- **Components:** metadata, link to intake status, link to review screen, activity log (sent date, submitted date)
- **Actions:** resend link, view review screen, archive engagement
- **Data shown:** status timeline

# 7. Data Model

**Read Section 1.5 first if you haven't** — the `AuditArea` and `kba_form_ref` fields below only make sense with that context, and they're the difference between this being a real scoping engine versus a generic form builder.

| Entity            | Key fields                                                                  | Notes                                              |
|-------------------|-----------------------------------------------------------------------------|----------------------------------------------------|
| <b>Firm</b>       | id, name, created_at                                                        | Supports multi-tenancy without building it out yet |
| <b>User</b>       | id, firm_id, email, password_hash, role (enum, only "firm_user" used in v1) | Role field reserved for v2 partner/senior split    |
| <b>Engagement</b> | id, firm_id, client_name, fiscal_year_end, entity_type                      | Central object tying everything together           |

| Entity            | Key fields                                                                                                                                                                                                                                                                                                                                                                                  | Notes                                                                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
|                   | (enum), client_type (enum), template_id, status (enum: draft/sent/in_progress/submitted/reviewed), created_at                                                                                                                                                                                                                                                                               |                                                                                                                                               |
| <b>EntityType</b> | id, name (church/school/nfp_program_income/nfp_general)                                                                                                                                                                                                                                                                                                                                     | Reference table, not user-editable in v1                                                                                                      |
| <b>ClientType</b> | id, name (first_year_new/continuing/re_audit/first_year_predecessor)                                                                                                                                                                                                                                                                                                                        | Reference table                                                                                                                               |
| <b>AuditArea</b>  | id, name (Cash, Investments, Revenue & Contributions, PP&E, AP & Disbursements, Payroll, Debt & Leases, Net Assets, Related-Party, UBI/Tax, etc.), kba_form_ref (e.g., "KBA-409"), aud_program_ref (e.g., "AUD-813"), is_always_in_scope (bool — true for Cash, Payroll, Net Assets, Financial Reporting/Closing; false for areas that depend on a scoping answer, like Investments or UBI) | <b>This is the new table.</b> It's a direct copy of KBA-400's Table 1 rows. Every question in the Scoping phase maps to exactly one row here. |
| <b>Template</b>   | id, firm_id (nullable for seeded system templates), name, entity_type_id, client_type_id, is_system_seeded (bool)                                                                                                                                                                                                                                                                           | A template = a question set + its rules                                                                                                       |
| <b>Question</b>   | id, template_id, phase (enum: preliminary/entity_understanding/fraud_inquiry/estimates/scoping/entity_level_controls/it_                                                                                                                                                                                                                                                                    | phase and kba_form_ref are new — they're what makes it possible to render the tabbed structure in the                                         |

| Entity      | Key fields                                                                                                                                                                                                                                                                                                                                     | Notes                                                                                                                                                                                                                                                                                                     |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             | general_controls/transaction_controls), kba_form_ref (e.g., "KBA-302", "KBA-400"), audit_area_id (nullable — only set for Scoping and Transaction Controls questions), text, answer_type (enum: yes_no/single_select/multi_select/short_text/number/date), options (json), order_index, parent_question_id (nullable), branch_condition (json) | prototype and to filter "only show Transaction Controls questions for AuditAreas that scoped in."                                                                                                                                                                                                         |
| Answer      | id, submission_id, question_id, value (json)                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                           |
| Rule        | id, template_id, trigger_question_id, trigger_condition (json), action_type (enum: add_request_item/add_flag/activate_audit_area), action_payload (json: request item text + planning area + guidance_note, or flag text + severity, or audit_area_id to activate), audit_area_id (nullable)                                                   | <b>activate_audit_area is a new action type.</b> A Scoping question's rule doesn't add one request item — it turns an entire <b>AuditArea</b> on or off, which is what then determines whether that area's Transaction Controls questions even render and whether its request items get generated at all. |
| RequestItem | id, submission_id, rule_id (nullable if auditor-added), audit_area_id, planning_area, text, guidance_note, is_auditor_added (bool), is_removed (bool), status (enum: requested/uploaded/received/needs_follow_up)                                                                                                                              | Now tied to <b>audit_area_id</b> — if an area never scoped in, no RequestItem rows exist for it. There's nothing to "hide," because nothing was generated.                                                                                                                                                |

| Entity            | Key fields                                                                                                                        | Notes                                                                                              |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Flag</b>       | id, submission_id, rule_id, text, severity (enum: info/missing_info/elevated_risk), resolved (bool)                               | Generated output                                                                                   |
| <b>Submission</b> | id, engagement_id, template_id, respondent_name, respondent_email, started_at, submitted_at, status (enum: in_progress/submitted) | One per intake send; supports resend/re-do                                                         |
| <b>Document</b>   | id, request_item_id, file_url (Supabase Storage path), file_name, uploaded_by, uploaded_at, note (optional)                       | One-to-many with RequestItem — a single item can have multiple uploaded files. No versioning in v1 |

---

## 8. Logic Engine

**Design principle: deterministic rules, not AI, for v1.** Every rule is: *if answer to Question X meets Condition Y, then do Action Z (add a request-list item, or raise a flag)*. This is auditable, explainable to a partner, and fast to build — which matters a lot for a regulated-adjacent workflow like audit planning, where "the AI decided that" is not an acceptable answer to a reviewer.

### Worked example: how scoping actually drives the request list (Whitfield Preparatory Academy)

This is the concrete version of the abstract rule above — walk through this before the generic explanation that follows, since it's what the prototype's Scoping tab demonstrates live.

Whitfield is a school, re-audited after a 2-year gap. During the **Scoping** phase (KBA-400), four questions get asked, one per **AuditArea**:

| Scoping question                                                | Answer   | Effect on <code>AuditArea.is_active</code>                                                                                             |
|-----------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------|
| Does the school hold investments or an endowment?               | No       | Investments → <b>inactive</b>                                                                                                          |
| Auxiliary/unrelated business income (rentals, bookstore, etc.)? | Not sure | Investments-adjacent UBI area → <b>conditionally active</b> (flag raised, area stays open pending confirmation)                        |
| Mortgage, bond, or lease on the facility?                       | No       | Debt & Leases → <b>inactive</b>                                                                                                        |
| Federal/state grant funding?                                    | Not sure | Revenue & Contributions stays <b>active</b> (it's always-in-scope for a school anyway), but a <code>missing_info</code> flag is raised |

Because Investments and Debt & Leases came back inactive:

- The **Transaction Controls** phase never shows a KBA-409 (Treasury) or debt-related question block to the respondent at all — those `Question` rows exist in the template but are filtered out at render time based on `AuditArea.is_active`.
- No `RequestItem` rows tied to `audit_area_id = investments` or `audit_area_id = debt` are ever created — not hidden, never generated.
- The auditor review screen shows exactly the areas that are relevant: Revenue & Contributions, PP&E, AP/Disbursements, Payroll, Net Assets, Financial Reporting/Closing — plus a "Needs confirmation" note next to UBI and grant funding until someone verifies those two "not sure" answers.

This is the entire point of the `activate_audit_area` rule type in Section 7 — it's a gate on a whole section of the questionnaire and the request list, not just one line item.

### How rules fire (step by step)

1. Each **Question** belongs to a **planning area** (Cash, Revenue/Contributions, Payroll, PP&E, AP/Disbursements, Governance, IT General Controls, Opening Balances).
2. Each **Rule** is attached to one question and a condition on its answer (e.g., "answer = Yes," "answer includes 'grants'," "answer is empty/blank").
3. When a rule's condition is met by a submitted answer, its **action** fires:

- **add\_request\_item**: appends a pre-written request-list line item (e.g., "Listing of all federal and state grants received or receivable, including grant agreements") tagged to the matching planning area.
  - **add\_flag**: appends a follow-up flag with a severity, so the auditor sees it separately from the routine list (e.g., "Client indicated cash flow is inadequate to meet obligations — evaluate going concern considerations before fieldwork").
4. Rules can also be **entity-type-conditioned** and **client-type-conditioned** at the template level — i.e., the same base question can trigger different request items depending on whether the template is "church" vs. "school," because the template itself is entity-specific (see Section 5, step 3).

### Example rule mappings, grounded in real planning practice

- **Client type = first-year with predecessor auditor** → automatically includes the predecessor-communication question block and, on submission, adds request items: prior-year financial statements, prior-year management letter, and a flag: "Obtain predecessor auditor authorization and complete predecessor communication before fieldwork begins," reflecting the standard opening-balance/predecessor-communication procedures for initial engagements.
- **Client type = first-year (any) or re-audit after a gap** → adds request items for opening balance support: prior-year trial balance, prior-year adjusting/reclassifying entries, and roll-forward support for beginning net asset balances.
- **Entity type = church** → adds church-specific request items (e.g., listing of in-kind contributions and donated services, contribution/pledge records, restricted fund listings) that would not appear for a school template.
- **Entity type = school** → adds school-specific items (e.g., tuition receivable aging, financial aid/Title fund grant agreements, enrollment data supporting tuition revenue).
- **Entity type = NFP with program income** → adds items tied to program service fee/contract revenue recognition (e.g., listing of program service contracts, performance-obligation documentation), reflecting the revenue-recognition and contribution-vs-exchange-transaction distinctions that actually drive audit work in this area.
- **"Going concern" style questions** (cash flow adequacy, loan covenant waivers/violations, deteriorating revenue trends, significant related-party transactions) — a "Yes" to any of these fires a **flag**, not just a request item, because these are the same red-flag questions used in client acceptance/continuance evaluation to signal elevated risk requiring auditor judgment, not a document.
- **"Client is unsure" answer on any question** → always fires a **missing\_info** flag rather than a silent skip, so nothing falls through branching logic unnoticed.
- **IT / general controls questions** (e.g., "does the entity use accounting software with user-level access controls," "has there been any system change or conversion this year") → route to an IT General Controls-tagged request item/flag, since a system conversion year is a known driver of additional planning procedures.

## The "intelligent tool" framing — where the intelligence actually lives

The tool should feel intelligent through the *quality and precision of the request list itself* — not through client-facing explanations. Each `add_request_item` action carries a `guidance_note`: a short, pre-written explanation of *why* that item is being asked for and *what it should include* (e.g., for "listing of in-kind contributions": *"Include donated goods, services, and use of facilities recognized in net assets — see your revenue recognition policy for the threshold used."*). **This note is auditor-facing only** — it appears in your review screen (6.7) as internal context/rationale for each item, but is *not* shown to the respondent in the client portal. The client sees the request item itself (what to upload) and nothing more; the reasoning behind it stays internal to the firm. This is stored as data (`Rule.action_payload.guidance_note`), so you can improve the wording over time without touching code, and it's fully consistent across every client that template applies to — it just isn't exposed outside the firm.

## Why this structure scales

Because rules are attached to (question + condition) pairs and templates are just bundles of questions + rules, adding a new entity type later (e.g., expanding beyond NFP) means writing a new template and its rules — it does not require new application code. This is the core "reusable across firms, industries, and audit types" property the vision calls for, achieved through configuration rather than a rebuild.

---

## 9. Prototype Build Recommendation

*(Written for someone who hasn't built software before — here's the reasoning behind each choice, not just the name of the tool.)*

The core decision: low-code/no-code backend + simple custom frontend, not full custom code

You don't need a custom backend written from scratch for an MVP like this. The data model in Section 7 is a set of tables with relationships — that's exactly what tools like **Airtable**, **Supabase**, or **Retool** are built for, and it lets your technical friend skip weeks of backend plumbing (auth, database setup, API endpoints) that add no unique value to *this* product. The unique value is the branching questionnaire UI and the rules engine — that's where custom code should go.

## Recommended stack

| Layer                                        | Recommendation                                                                                                                                  | Why                                                                                                                                                                                                                                                                                |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Database + backend                           | <b>Supabase</b> (Postgres + built-in auth + auto-generated APIs)                                                                                | Free tier is enough for a prototype; gives you a real relational database (matches Section 7 exactly) instead of outgrowing a spreadsheet-like tool; built-in login/auth so you don't build that from scratch                                                                      |
| Frontend (respondent form + auditor screens) | <b>Next.js (React) + Tailwind CSS</b> , deployed on <b>Vercel</b>                                                                               | Standard, well-documented, huge amount of free tutorials/AI-assistance available if your friend gets stuck; Vercel deployment is literally a git push                                                                                                                              |
| Auth                                         | Supabase Auth (email/password or magic link)                                                                                                    | Comes free with the database choice above — no separate service needed                                                                                                                                                                                                             |
| Rules engine                                 | Plain application logic (JavaScript/TypeScript functions) reading the <b>Rule</b> table and evaluating conditions against <b>Answer</b> records | Don't use a third-party "workflow automation" tool for this — it needs to be fast, testable, and fully under your control since it's the core IP records                                                                                                                           |
| File storage                                 | <b>Supabase Storage</b> (buckets, access-controlled per engagement)                                                                             | Comes free with the same Supabase account as the database/auth — no separate storage vendor to integrate. Keep it simple in v1: one bucket, files organized by <b>engagement_id/request_item_id/</b> , access restricted to the firm user and the specific engagement's respondent |



| Layer                   | Recommendation                                                                                                      | Why                                    |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| <b>Export (PDF/CSV)</b> | A PDF generation library (e.g., <code>react-pdf</code> or a server-side PDF library) + simple CSV string generation | Both are well-trodden, low-risk pieces |
| <b>Deployment</b>       | Vercel (frontend) + Supabase (backend/db), both free at prototype scale                                             | No DevOps knowledge required to start  |

## Should AI be built in now, or deferred?

**Defer it.** The MVP's value proposition is explicitly *not* "AI for audit" — it's a standardized, explainable rules engine. Adding an LLM into the request-list generation now would (a) make output non-deterministic and harder for a partner to trust, (b) add cost and complexity you don't need to prove the core loop works, and (c) contradict the `important_constraints` you set. The right place for AI later (v2+) is drafting a planning memo *from* the already-structured, already-trusted answers and request list — a summarization task layered on top of a reliable core, not a replacement for it.

## No-code vs. low-code vs. full-code — the actual tradeoff

- **Pure no-code (e.g., staying on Jotform, or using Airtable+Softtr):** fastest to a demo, but you already tried Jotform and hit its ceiling — branching + a real rules engine + a clean auditor review UI is exactly where no-code tools get clunky and hard to extend.
- **Low-code backend + custom frontend (recommended above):** best tradeoff for this project — you get a real database and real UI control without your friend hand-building auth, hosting, and infrastructure from zero.
- **Full custom code (own backend framework, own hosting):** appropriate later, once you have paying customers and specific scaling/security requirements (e.g., SOC 2) that outgrow Supabase — not needed to validate the idea.

---

## 10. MVP Build Plan

### Phase 0 — Prototype (4–6 days)

- Set up Supabase project with the Section 7 schema (core tables: Engagement, Template, Question, Answer, Rule, RequestItem, Flag, Submission, Document)
- Hard-code **one** seeded template (e.g., "NFP General — Continuing Client") with ~15–20 questions, ~15 rules, and guidance notes written on each request-item rule

- Build the respondent intake form (no branching yet, just linear), the auditor review screen showing the generated list, and a bare-bones upload control per request item (one file, no status workflow yet — just "file present or not")
- No auth yet, or the simplest possible login — single hardcoded firm user + single hardcoded respondent
- **Goal:** prove the core loop end to end — answers in, structured list with guidance out, at least one document uploaded against an item

## Phase 1 — MVP (5–7 weeks)

- Week 1–2: Add real auth for the firm user and a lightweight respondent login (single firm, matches Section 1), multi-engagement dashboard, entity type/client type selection, template auto-suggestion
- Week 2–3: Build branching logic in the intake form and the template builder screen (Section 6.5) so you can add templates without touching code
- Week 3–4: Build out the remaining 3 seeded templates (church, school, NFP-with-program-income) with real planning content and guidance notes (Section 8 examples are your starting library)
- Week 4–5: Flags panel, PDF/CSV export, save-and-resume for respondents
- Week 5–6: Full document status workflow (Requested/Uploaded/Received/Needs Follow-up), the Client Document Portal screen (6.9), auditor can mark items received/needs-follow-up
- Week 6–7: Auditor can edit/add/remove request items before finalizing; polish, test with 2–3 real engagements internally

## Phase 2 — v2 (later, scope by demand)

- Document versioning and e-signature
- Partner review/sign-off workflow (role separation)
- AI-drafted planning memo from submitted answers
- Multi-firm onboarding (only needed once you're selling beyond your own firm)
- Analytics dashboard across engagements

**Minimum build sequence if you had to cut further:** Phase 0 loop (including basic upload) → auth + dashboard → one full template with branching → full document status workflow → export. Everything else (additional templates, editing, save/resume) can ship incrementally after the first real client uses it.

---

## 11. Risks and Simplifications

**What could make this too complex:**

- Trying to make the rules engine fully generic/configurable for *any* industry from day one, instead of nonprofit-specific. Resist this — hard-code nonprofit planning-area categories and expand later.
- Over-building the document exchange feature: versioning, e-signature, in-app commenting, and file preview are all real portal features eventually, but none are needed to prove "client uploads a file against the right request item and the auditor sees it." Keep v1 to upload + status only (see Section 3 note).
- Building partner/reviewer sign-off workflows before you have more than one user per firm using the tool.
- Trying to support every possible question/answer type (matrix questions, ranking, etc.) — five basic types (Section 3) cover the actual planning questions in KBA-style intake forms.

### **What should be simplified:**

- Template creation for v1: it's fine if only *you* (as the builder/domain expert) create and edit templates directly in the database or a very bare-bones admin screen, rather than building a polished template-builder UI (Section 6.5) on day one. Promote that screen to should-have if it slows down Phase 0/1.
- Branching logic: start with simple "show question B only if answer to question A = X" — don't build multi-condition branching (AND/OR trees) until you've seen real templates need it.
- Document status: four states (Requested/Uploaded/Received/Needs Follow-up) is enough — resist adding more granular states until real use shows a gap.

### **Reasonable assumptions to ship faster:**

- Single currency, single language, US-only regulatory framing (matches your target niche).
  - One respondent per submission (no multi-contact collaborative form-filling in v1).
  - Request list text and guidance notes are pre-written per rule (from your audit knowledge), not dynamically generated — this keeps output predictable and easy for a partner to trust.
  - Single firm for the prototype (per Section 1) means no need to build firm signup, billing, or multi-tenant admin yet — every table still carries `firm_id` so this isn't a rewrite when you're ready to onboard firm #2.
-

## 12. Monetization and Positioning

**First niche to sell into:** small-to-mid CPA firms performing not-for-profit audits — specifically firms with **5–15 recurring NFP engagements per year**, where the repeatability of the workflow (same rough entity types, same seniors doing intake repeatedly) makes the time savings obvious and provable. This is narrower than "all CPA firms," which is the point — a tight niche lets you build templates that are genuinely good rather than generically shallow.

**Pricing approach:** Per-firm subscription (not per-engagement or per-seat, at least initially) — firms think in terms of "a tool I use across my book of clients," and per-engagement pricing creates friction discouraging trial use on smaller/simpler clients where the value is still real. A reasonable MVP starting point is a flat monthly fee with a cap on active engagements per month, with a higher tier removing the cap — but treat this as a hypothesis to validate with your first 3–5 paying firms, not a fixed decision.

**How this differs from generic onboarding software (Typeform, Jotform, generic intake tools):** those tools collect answers; they don't encode audit-specific logic (entity type × client type × planning area) into a request list an auditor already trusts the structure of. The value isn't the form — it's the pre-built professional judgment baked into the rules engine, which a generic form tool has no concept of.

**How this differs from full audit platforms (Caseware, AuditBoard, Workiva):** those are comprehensive audit execution systems (working papers, sampling, sign-off chains) that are expensive, heavy to implement, and overkill for the single problem this MVP solves. This product is intentionally a **narrow point solution** for the intake-to-request-list step — a firm can adopt it in a day without an implementation project, and it can plausibly integrate *alongside* those larger systems later rather than compete with them directly.

---

## 13. Final Builder Brief (hand this section to your developer)

**Read this document in this order, not top to bottom:** Section 1.5 (Audit Methodology Primer) first — it's the context that makes everything else non-arbitrary. Then this section. Then open [audit-portal-prototype.html](#) directly in a browser (double-click it, no server needed) and click through it — specifically the Client view → Planning questionnaire → Scoping tab, then toggle an answer and check the Transaction Controls tab. That interaction is worth more than a paragraph of explanation: it's the actual mechanic the data model in Section 7 is built to support. Sections 2–12 are the surrounding product/business context; skim them, but 1.5 → 13 → prototype is the fast path to being productive.

**Project:** A portal where a CPA firm sends a branching client questionnaire to a nonprofit audit client (church, school, or NFP with program income); answers are automatically converted into a categorized, guidance-annotated document request list; the same portal lets the client upload documents against each request-list item and lets the auditor track receipt status. Single firm for the prototype.

**Stack:** Supabase (Postgres + Auth + Storage) for backend/database/files, Next.js + Tailwind on Vercel for frontend. No AI/LLM calls in v1 — the "intelligence" is deterministic rule evaluation plus pre-written guidance text against a **Rule** table, described in Section 8 above.

**Data model:** See Section 7 — 14 core tables, including **AuditArea**, which is the table that matters most (it's a direct copy of KBA-400's audit-area checklist — see Section 1.5). Build this schema first; it's the contract for everything else.

**Core loop to build first (Phase 0, ~1 week):**

1. Firm user creates an Engagement with entity\_type + client\_type.
2. Respondent logs into the portal and answers the **Scoping** phase questions first (one per **AuditArea**) — these use the **activate\_audit\_area** rule type to flip **AuditArea.is\_active**.
3. The **Transaction Controls** phase then only renders **Question** rows whose **audit\_area\_id** points to an active **AuditArea** — inactive areas are filtered out entirely, not just hidden with CSS. This is the piece worth getting right first; everything else is a more familiar CRUD pattern.
4. On submit, the app evaluates every Rule tied to that Template against the submitted Answers, and writes RequestItem (with guidance\_note, and tagged to its **audit\_area\_id**) / Flag records — only for areas that scoped in.
5. Auditor review screen displays RequestItems grouped by planning\_area with their guidance notes (auditor-only — never shown to the respondent), and Flags in a separate panel with severity + the reason (which rule/answer triggered it).
6. Respondent can upload at least one file against a RequestItem; auditor can see that a file was uploaded.

**Then add, in order:** auth/dashboard → branching logic → remaining 3 seeded templates → full document status workflow (Requested/Uploaded/Received/Needs Follow-up) → export (PDF/CSV) → auditor edit-before-finalizing → save-and-resume.

**Explicitly not in v1:** document versioning/e-signature, partner sign-off workflow, AI-generated content of any kind, multi-industry templates beyond nonprofit, multi-firm billing/onboarding.

**Source of truth for what the request-list rules and guidance notes should actually say:** the domain owner (CPA, non-technical) will supply the question text, branching logic, and

request-list/flag/guidance content per template — the developer's job is the engine, storage, and UI that executes it, not the audit content itself.